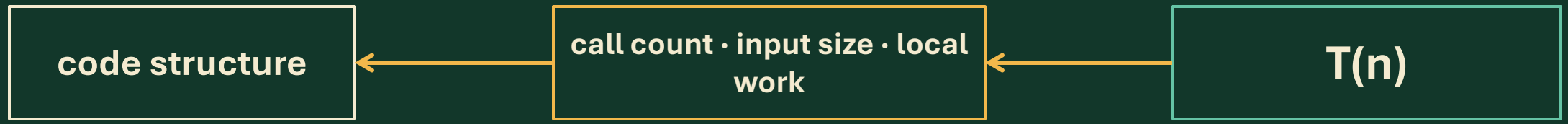
Why recursion and recurrences?

PROGRAM STRUCTURE



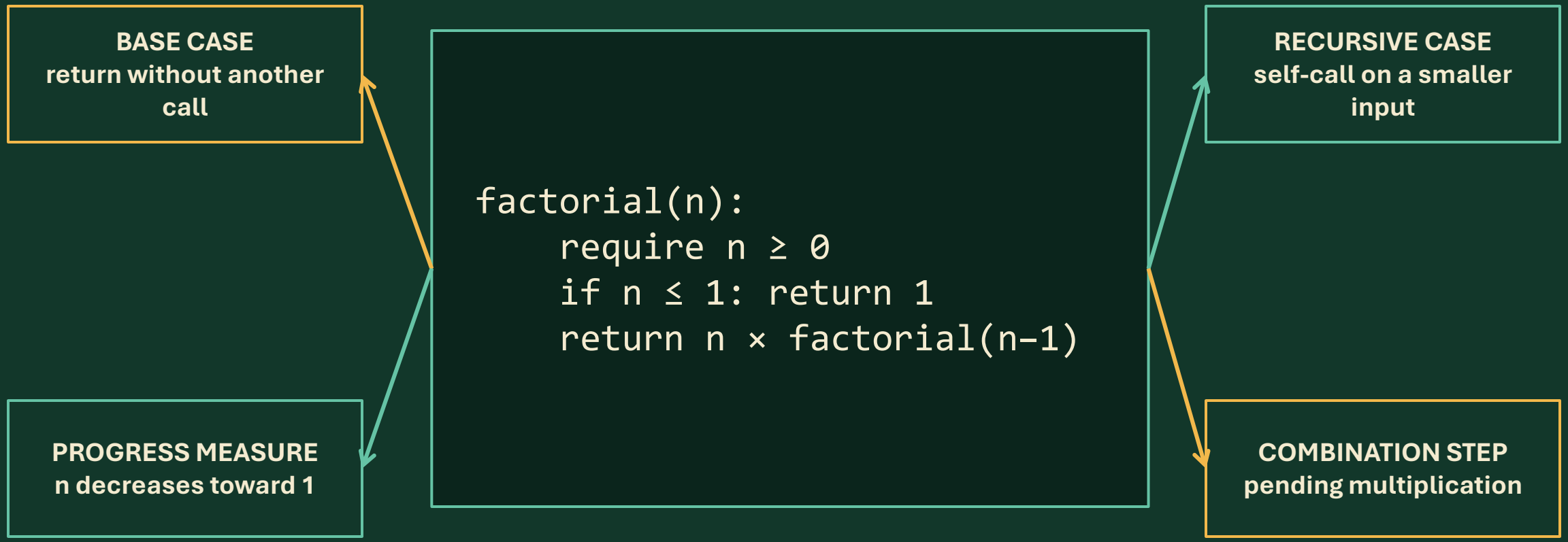
COST MODEL



Binary search halves the remaining interval.

Recursion describes calls. A recurrence describes cost.

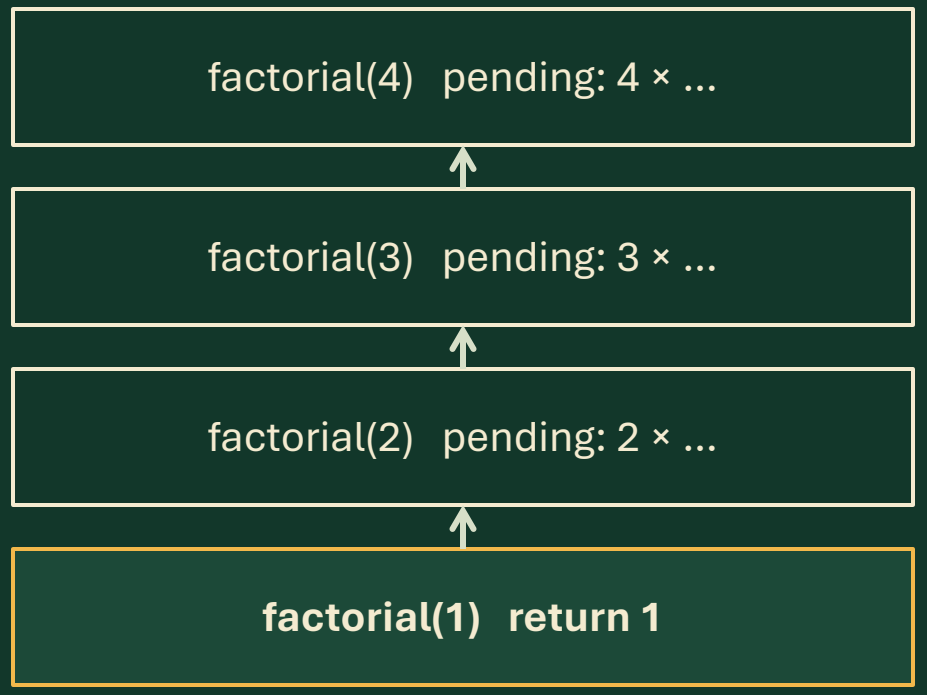
Anatomy of a recursive function



Correctness begins with the input contract.

The call stack

DESCENT — active frames



UNWIND — pending work resumes

1

↑

2 × 1 = 2

↑

3 × 2 = 6

↑

4 × 6 = 24

Maximum active depth: 4 frames

Termination is an argument

valid input + reachable base case + decreasing measure ⇒ termination argument

INPUT DOMAIN	MEASURE	UPDATE	BASE CONDITION
$n \in \mathbb{N}_0$	$m(n)=n$	$n \leftarrow n-1$	$n=0$

PROMPT: what is wrong here?

```
countdown(n):
    if n == 0: print("done")
    countdown(n)
```

“It looks smaller” is not a proof.

Mathematical termination ≠ runtime stack capacity

How to write a recurrence

Inspect the algorithm before writing the equation.

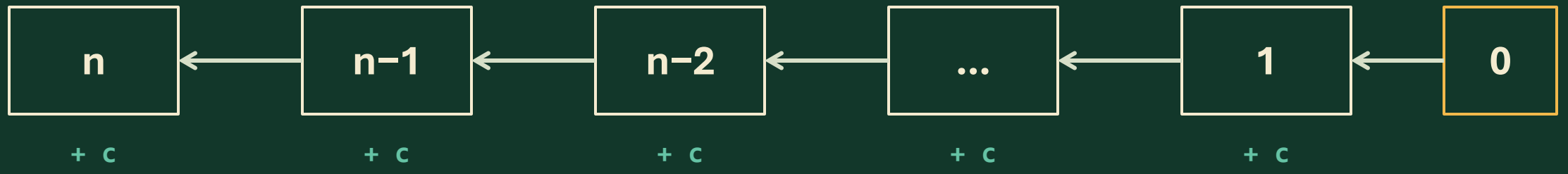
RECURSIVE CALLS	SUBPROBLEM SIZES	COMBINE WORK
a calls	n / b each	f(n)
2 calls	**n / 2 each**	**Θ(n)**

$$T(n) = 2T(n/2) + \Theta(n)$$

Always state a base condition: $T(1)=\Theta(1)$

Expansion: T(n) = T(n−1) + Θ(1)

$$T(n) = T(n-2) + 2\Theta(1) = \dots = T(0) + n\Theta(1)$$



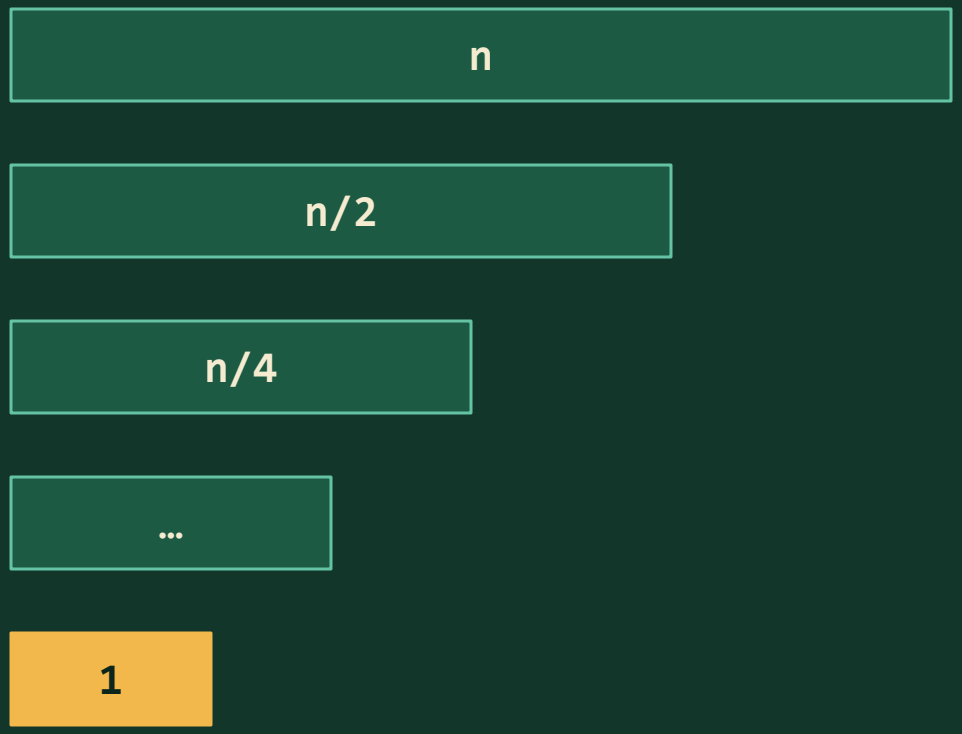
n levels × constant work per level

$$\Theta(n)$$

Depth: Θ(n) · Auxiliary stack: Θ(n)

Halving gives logarithmic depth

SUBPROBLEM SIZE



$$n / 2^k \leq 1$$

$$\Rightarrow$$

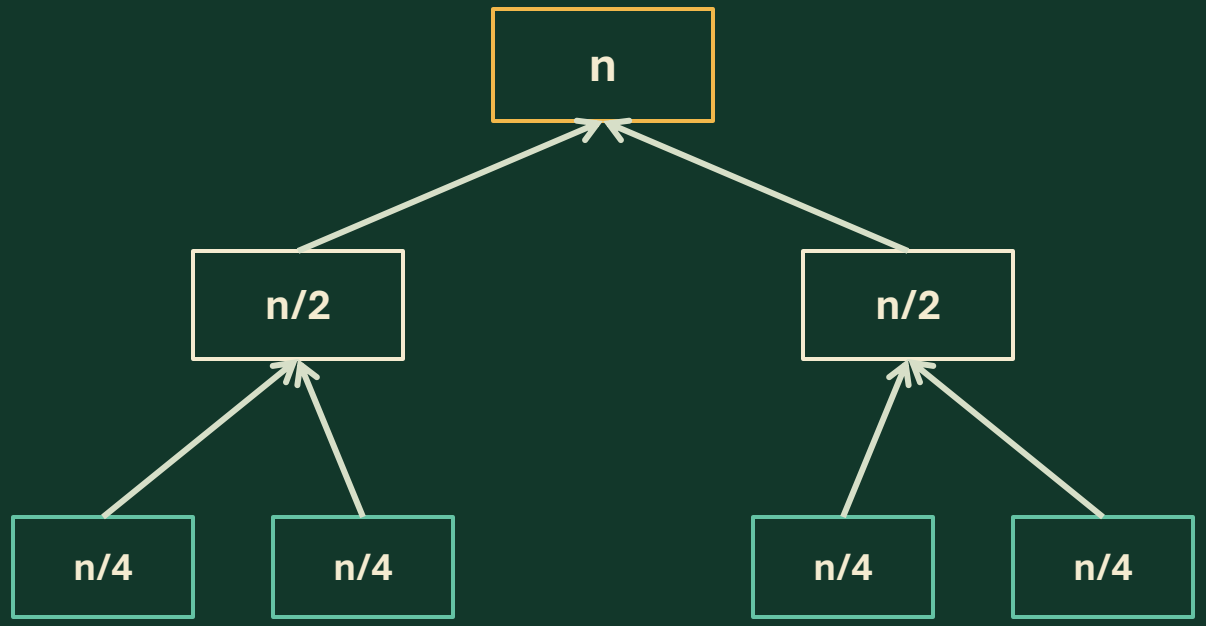
$$k \geq \log_2 n$$

For n = 64, depth = 6 halvings

One recursive branch → logarithmic levels

Recursion tree: two halves + linear combine

$$T(n) = 2T(n/2) + \Theta(n)$$



LEVEL	TOTAL
cn	$\Theta(n)$
$2 \cdot c(n/2)$	$\Theta(n)$
$4 \cdot c(n/4)$	$\Theta(n)$

$\Theta(\log n)$ levels × $\Theta(n)$ per level

$$T(n) = \Theta(n \log n)$$

Master Theorem: shape and comparison

$$T(n) = aT(n/b) + f(n)$$

$$p = \log_b a$$

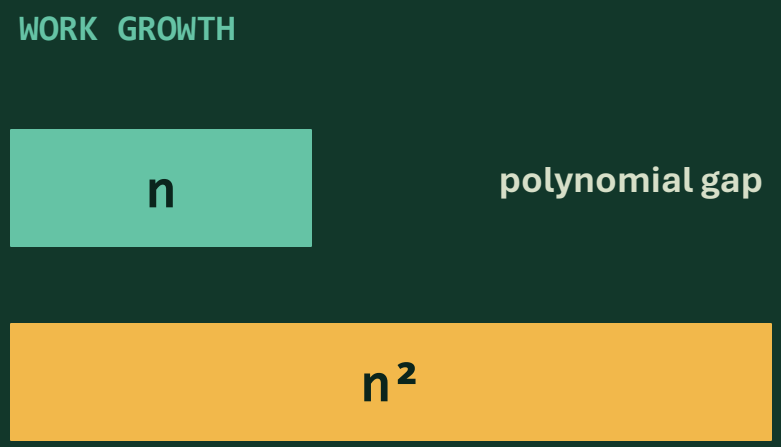
RECURSIVE-DOMINATED	BALANCED	NON-RECURSIVE-DOMINATED
f(n) polynomially smaller	f(n) matches n^p	f(n) polynomially larger
n^p wins	**add a log factor**	**f(n) wins + regularity**

Compute p first. Then compare. Then check conditions.

Case 1: recursive work dominates

If $f(n)=O(n^{p-\varepsilon}),\ \varepsilon>0$

then $T(n)=\Theta(n^p)$

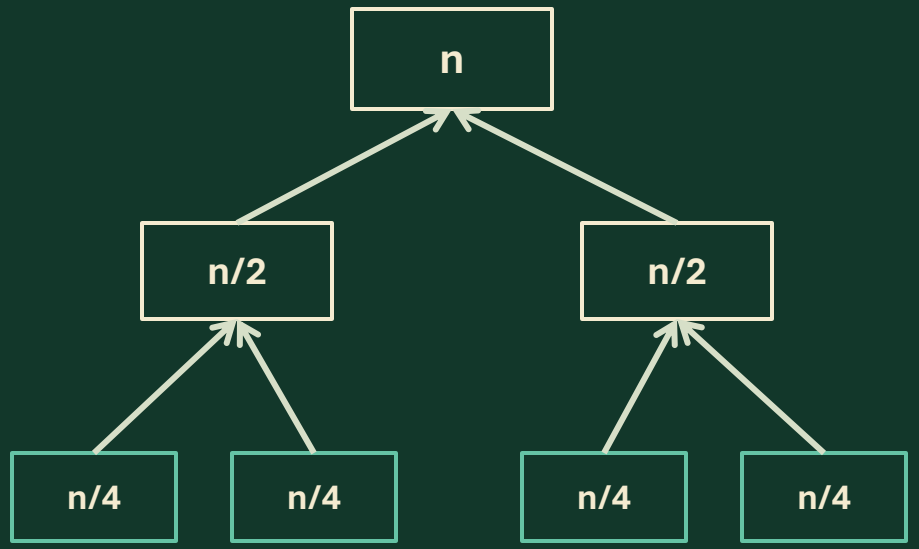


$4T(n/2) + \Theta(n)$ $\quad p = \log_2 4 = 2 \quad \Rightarrow \quad \Theta(n^2)$

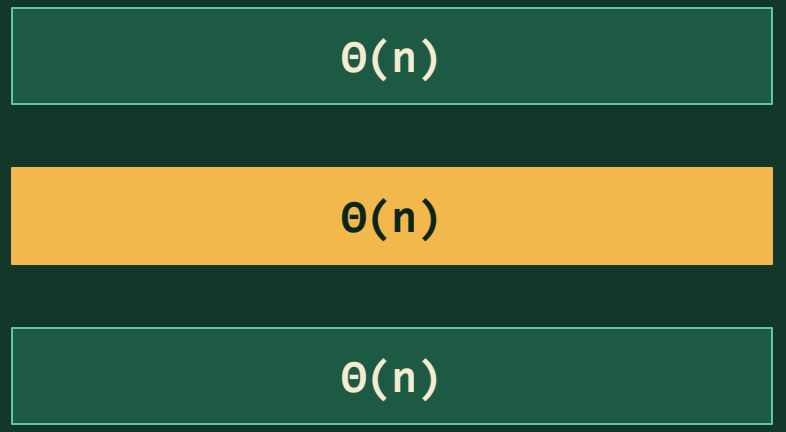
Leaves dominate lower-order combine work.

Case 2: balanced work

$$f(n)=\Theta(n^p \log^k n) \Rightarrow T(n)=\Theta(n^p \log^{k+1} n)$$



EQUAL WORK PER INTERNAL LEVEL



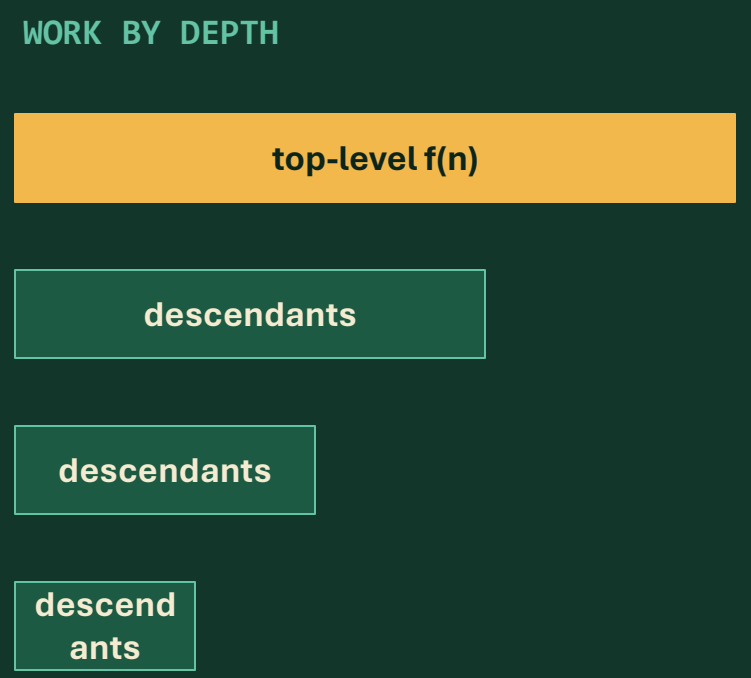
Θ(log n) levels

Merge sort: Θ(n log n)

Case 3: combine work dominates

1. $f(n)=\Omega(n^{p+\varepsilon}),\quad \varepsilon>0$
2. $a\,f(n/b) \le c\,f(n),\quad c<1$

then $T(n)=\Theta(f(n))$

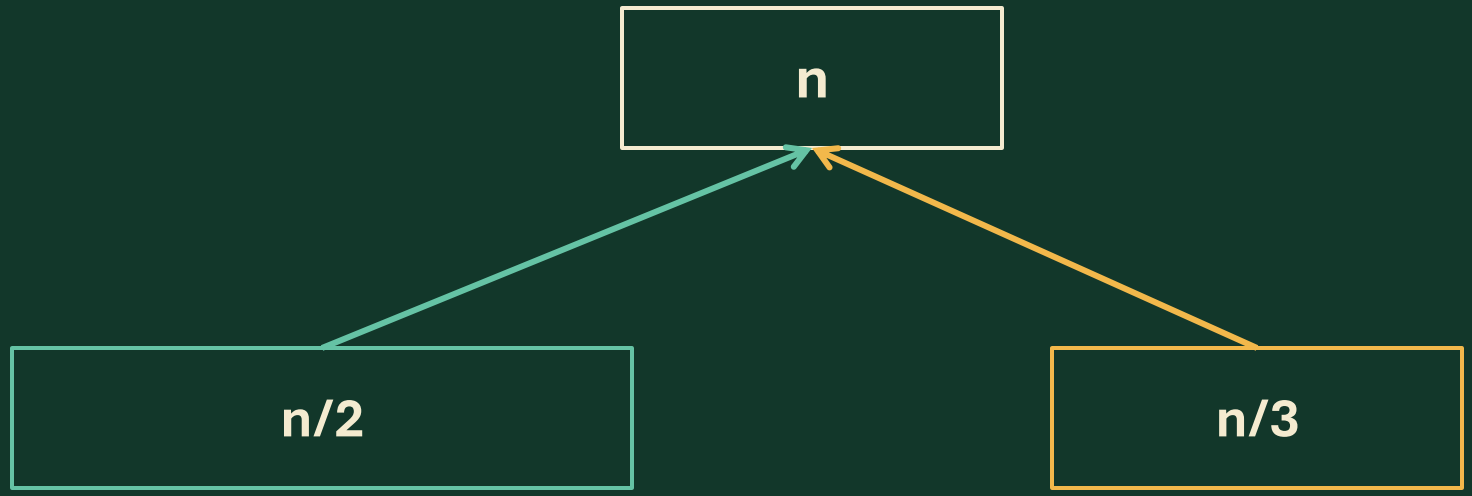


$2T(n/2)+\Theta(n^2)$ $\quad p=1$ $\quad \Rightarrow \Theta(n^2)$

"f is bigger" is not the full theorem statement.

When the basic theorem does not fit

$$T(n) = T(n/2) + T(n/3) + n$$



different subproblem widths

$$\neq \quad aT(n/b)+f(n) \text{ with equal-size subproblems}$$

Use a recursion tree, substitution, or a more general theorem.

Binary search contract

PRECONDITION

Input must be sorted

INTERVAL

[lo, hi)



Return a matching index — or −1

`[0,0)` represents an empty array naturally

Trace a successful search

target = 13



STATE	lo	hi	mid	COMPARISON
[0,7)	0	7	3	9 < 13
[4,7)	4	7	5	18 > 13
[4,5)	**4**	**5**	**4**	**13 = 13 → found**

```
mid = lo + (hi - lo) // 2
```

The loop invariant

If the target occurs, an occurrence remains inside [lo, hi).



sortedness justifies each exclusion

`progress measure: hi - lo` — each update makes it smaller

At lo = hi, the candidate interval is empty.

Off-by-one defect

If values[mid] < target, the midpoint is ruled out.



WRONG

`lo = mid → [1,2) → mid = 1 again`

RIGHT

`lo = mid + 1 → [2,2) → interval empty`

A consistent boundary convention guarantees shrinkage.

Time and auxiliary space

METHOD	WORST-CASE TIME	EXTRA SPACE	ASSUMPTION
Linear search	$\Theta(n)$	$\Theta(1)$	none
Binary search · iterative	$\Theta(\log n)$	$\Theta(1)$	sorted random access
Binary search · recursive	$\Theta(\log n)$	$\Theta(\log n)$	sorted random access

Best case: $\Theta(1)$ when the first midpoint matches

Sorting first can cost $\Theta(n \log n)$: query count and update frequency matter.

Practical: test boundaries

BOUNDARY CHECKLIST

- ☐ empty array
- ☐ singleton: hit / miss
- ☐ first and last element
- ☐ missing: below / between / above
- ☐ **duplicates: document policy**

```
# Add one boundary case
cases = [
    ([], 5),
    ([7], 7),
    ...
]

# When a test fails, record:
# lo   hi   mid   values[mid]
```

RUN → MINIMISE → TRACE → LOCATE THE INVARIANT VIOLATION

Paper trace is the offline fallback.

Exit ticket

01 Solve one supported recurrence.

02 State the binary-search invariant.

03 Name one Master-Theorem limitation.

Write assumptions, not only the final answer.